

# DeepOHeat con k in input

Diario degli esperimenti, parte 3: un modello per tutte le configurazioni di conduttività. 31 agosto 2026.

## Il problema in due parole

Stesso chip a due materiali della parte 2, ma con una differenza di fondo (PDF `k_trainable.pdf`): nella parte 2 il campo di conduttività era **noto a priori** e il modello era costruito su quell'informazione ( $k_1$ ,  $k_2$  e l'interfaccia dentro la loss, la feature  $|y - y_i|$  nel trunk). Adesso il campo  $k$  **non è più cablato nel modello**: entra come input, tramite un secondo branch il cui output moltiplica (prodotto di Hadamard) quello del branch della potenza, come nello schema del PDF. Un solo modello deve servire tutte le configurazioni: materiali diversi ( $k$  in 0,3–2) e interfaccia in posizione diversa ( $y_i$  in 0,2–0,8).

Il training resta physics-informed: nessuna soluzione di training, a ogni batch si campionano ( $k_1$ ,  $k_2$ ,  $y_i$ ) e la loss FD conservativa della parte 2 usa il  $k$  del campione (media armonica sulle facce  $y$ ). Tutto ciò che aveva funzionato è ereditato intatto: `pin_level` (la media del fondo è  $a + b \cdot \langle f \rangle$  per bilancio energetico, **indipendente da  $k$** : il vincolo resta esatto per qualsiasi configurazione), augmentation di ampiezza, input a flusso esatto, boundary-layer check.

## Passo 0 — Un riferimento per k arbitrari: il solutore FD

Con  $k$  in input il modello va validato su  $k$  *diversi*, ma la ground truth Ansys esiste per una sola configurazione (i casi 11 e 14). Era la prima "direzione con margine" della parte 2: un solutore diretto nostro. `new_experiment/fd_solver.py` risolve  $\text{div}(k \text{ grad } u) = 0$  con le stesse BC della loss (stencil conservativo ai nodi, pesi di volume ai bordi, matrice simmetrica, AMG + CG): **~2 secondi per caso** a  $101 \times 101 \times 51$ . Validazione contro Ansys:

| caso            | input                                | rel L2 vs Ansys | max err | offset medio |
|-----------------|--------------------------------------|-----------------|---------|--------------|
| 11 (rettangolo) | f esatta ( $k \cdot u_z$ dai dati)   | 0,24%           | 0,08 °C | -0,01 °C     |
| 11              | mappa $21 \times 21$ a flusso esatto | 1,51%           | 0,28 °C | +0,10 °C     |
| 14 (quadrante)  | f esatta ( $k \cdot u_z$ dai dati)   | 0,94%           | 0,22 °C | -0,09 °C     |
| 14              | mappa $21 \times 21$ a flusso esatto | 0,70%           | 0,23 °C | -0,07 °C     |

Con  $f$  esatta il disaccordo è 0,1–0,2 °C: il livello degli artefatti del riferimento Ansys stesso (generato su  $21 \times 21 \times 11$  e interpolato). Test set (`make_k_testsets.py`): i 2 casi Ansys più **12 casi con 6 configurazioni di  $k$  mai viste** (interfaccia a 0,30 e 0,72, materiali invertiti,  $k$  uniforme) su 2 mappe di potenza held-out.

## Il modello: un branch per k

`new_experiment/models_k.py` — il branch  $k$  è un MLP piccolo ( $21 \rightarrow 64 \rightarrow \dots \rightarrow r$ ) che riceve il profilo  $k(y)$  sui 21 punti sensore, in scala logaritmica; nella cella a cavallo dell'interfaccia il valore è la **media pesata per la frazione di volume**, così  $y_i$  è ricostruibile esattamente dall'input anche tra i sensori (lezione della parte 2: l'input conta quanto il modello). Il suo output moltiplica elemento per elemento quello del branch della potenza, e l'einsum separabile resta identico.

Per il gomito, che ora può stare ovunque, l'idea di partenza era un **dizionario di kink**: trunk  $y$  con  $[y, |y - c_1|, \dots, |y - c_{17}|]$  su una griglia fissa di centri, con il branch  $k$  a pesare le componenti. Spoiler della sezione risultati: **non serve**.

## Risultati

| run                                               | Ansys 11     | Ansys 14     | 12 k mai visti | note                             |
|---------------------------------------------------|--------------|--------------|----------------|----------------------------------|
| k fisso (sanity, 30k ep)                          | 0,65%        | 1,36%        | 2,55%          | k fuori distribuzione nel 3° set |
| solo y_i variabile (30k)                          | 1,06%        | 5,39%        | 2,26%          |                                  |
| famiglia completa (50k)                           | 1,20%        | 5,55%        | 1,69%          |                                  |
| completa senza dizionario (30k)                   | 1,24%        | 5,61%        | 1,64%          | identico: dizionario inutile     |
| <b>completa, 60k ep, decay 1200, branch k 128</b> | <b>1,15%</b> | <b>5,20%</b> | <b>1,56%</b>   | <b>max err medio 0,87 °C</b>     |

- **Sanity superata:** a parità di problema (k fisso) il modello a due branch rifà i numeri della parte 2 (0,65/1,36 contro 0,72/1,33). Il secondo branch non costa nulla.
- **La generalizzazione su k funziona:** 1,56% medio su 12 configurazioni mai viste (il modello a k fisso, sugli stessi casi, fa 2,55%). Interfacce a 0,30 e 0,72, materiali invertiti e k uniforme: forma corretta ovunque, casi facili sotto l'1%.
- **Il dizionario di kink è inutile con la loss FD:** l'ablation senza dà numeri identici con metà epoche. Il trunk liscio approssima il gomito da solo; era la PINN a collocazione a rendere indispensabile la feature  $|y-y_i|$  (300× sul termine d'interfaccia). I run successivi girano senza: modello più semplice.

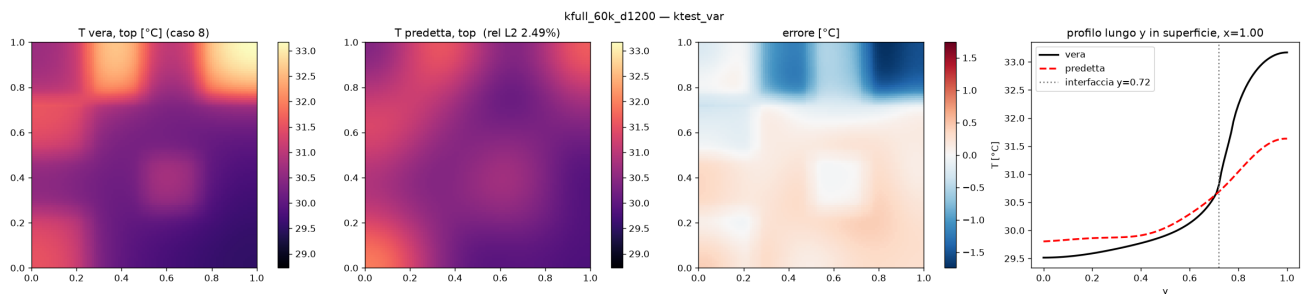


Fig. 1 — Modello finale sul caso peggiore dei 12 mai visti ( $k = 0,7/0,35$ , interfaccia a  $y = 0,72$ ): il gomito è al posto giusto anche dove il modello non ha mai visto un'interfaccia in quel punto con quel contrasto.

## La tasso di condizionamento e il caso 14

Il prezzo del modello unico: sullo stesso  $k$  della parte 2, il rel L2 raddoppia ( $0,9 \rightarrow 2,0\%$  sui casi FD) e sul caso Ansys 14 quadruplica ( $1,36 \rightarrow 5,2\%$ ). La diagnosi esclude il livello: `p1n_levell` tiene (offset  $-0,13$  °C, uguale al modello dedicato). L'errore è un **tilt lungo y**:  $-0,8$  °C sul lato  $k_1$ ,  $+0,7$  °C sul lato  $k_2$ , zero vicino all'interfaccia — il modello che serve tutti i  $k$  ripartisce leggermente male il salto termico tra i due materiali, e paga di più sulla mappa più atipica (il quadrante appoggiato alle pareti).

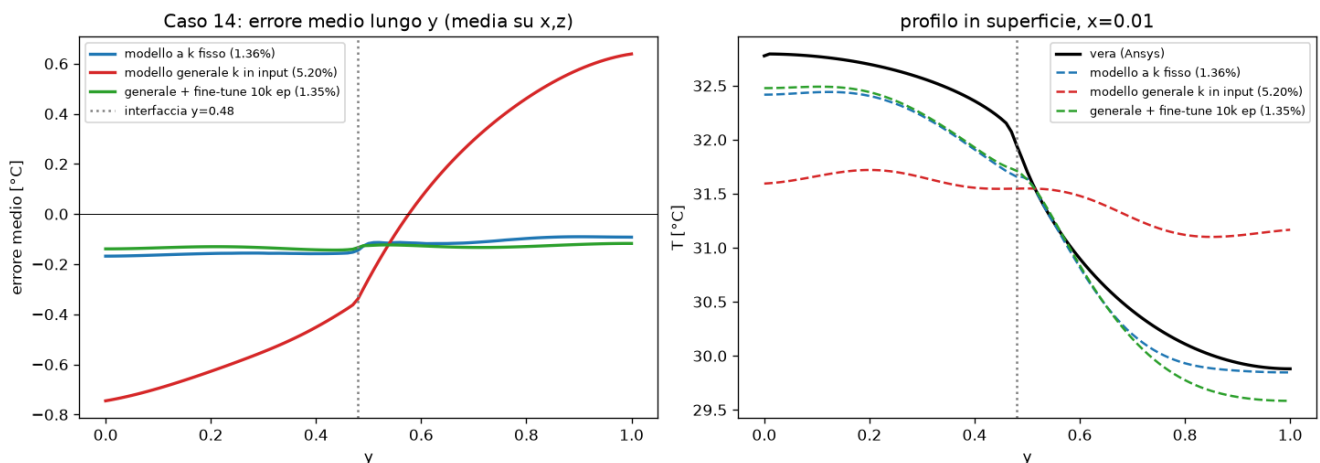


Fig. 2 — Caso 14. Sinistra: errore medio lungo  $y$  — il modello a  $k$  fisso (blu) è piatto, il modello generale (rosso) ha il tilt a cavallo dell'interfaccia; il fine-tune (verde) lo azzerava. Destra: profilo di temperatura in superficie.

Due ipotesi verificate e **smentite**: (1) la quantizzazione dell'interfaccia sulla griglia FD della loss (passo 0,025, bias variabile da campione a campione) — corretta con la media armonica pesata per la frazione di segmento sulla faccia a cavallo: numeri identici (5,55 vs 5,61%); la modifica resta perché è fisica più corretta a costo zero. (2) Il learning rate: con decay 600 il training muore a ~30k epoche; a 60k epoche con decay 1200 il guadagno c'è ma è marginale (1,69 → 1,56%). Un secondo seed misura il rumore: ~0,1% di rel L2.

## Il rimedio che funziona: fine-tune a k fisso

Se a inferenza la configurazione è nota, 10k epoche a k fisso (lr 3e-4) partendo dai pesi del modello generale — flag `--init_from` — recuperano **tutta** la tasso di condizionamento in **107 secondi**:

| modello                              | Ansys 11     | Ansys 14     | costo        |
|--------------------------------------|--------------|--------------|--------------|
| dedicato, allenato da zero (parte 2) | 0,72%        | 1,33%        | ~6 min       |
| generale (k in input)                | 1,15%        | 5,20%        | —            |
| <b>generale + fine-tune 10k ep</b>   | <b>0,60%</b> | <b>1,35%</b> | <b>107 s</b> |

Flusso di lavoro consigliato: il modello generale per esplorare materiali e posizioni dell'interfaccia, il fine-tune lampo quando la configurazione è decisa. Inseguire il tilt residuo con più capacità o famiglie di mappe ad hoc ha ROI basso: sotto ~0,5 °C si fittano gli artefatti del riferimento (lezione della parte 2).

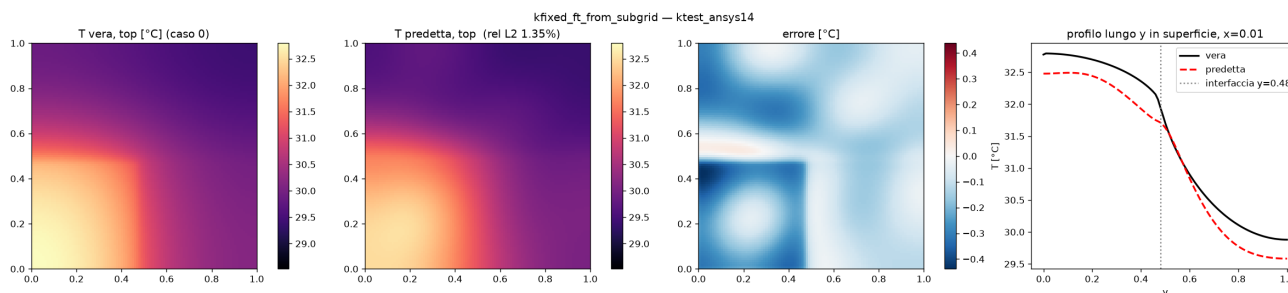


Fig. 3 — Caso 14 dopo il fine-tune: rel L2 1,35%, errore massimo 0,44 °C. Il tilt della Fig. 2 è sparito.

## Cosa abbiamo imparato

- Lo schema del PDF (secondo branch + Hadamard) funziona così com'è: la parte difficile non era l'architettura ma **avere un riferimento** per k arbitrari (il solutore) e capire dove finisce la capacità del modello unico (il tilt).
- La loss FD conservativa generalizza a k variabile quasi gratis: k per campione e media armonica sulle facce. Nessun termine d'interfaccia ad hoc, per nessuna posizione dell'interfaccia.
- `pin_level` è indipendente da k: il livello resta fissato dalla fisica anche quando il materiale cambia a ogni campione.
- Il dizionario di kink era un'ipotesi ragionevole e sbagliata: con la fisica definita sui punti FD, il trunk liscio basta. Vale il rasoio: feature in meno, modello più semplice.
- Un'ora di GPU non serve: tutti i run di questo report sono CPU, 7–14 minuti l'uno (11 ms/iterazione).
- Il fine-tune da modello generale è più veloce e leggermente migliore del modello dedicato da zero: il generale è un buon prior.

## Cosa NON dimostrano questi esperimenti

- I 12 casi di test sono generati con lo **stesso stencil** della loss (griglie diverse: 101 vs 41): un errore sistematico comune a entrambi non si vedrebbe. I casi Ansys restano l'unico controllo esterno, e coprono un solo k.

- La famiglia è due materiali stratificati lungo y. Il branch k accetta qualsiasi profilo, ma più strati, gradienti continui o  $k(x,y,z)$  pieno non sono stati né allenati né testati.
- Le mappe f dei test a k variabile sono blocchi: il tilt del caso 14 suggerisce che la copertura di mappe "a parete" nel training set conti; non è stato isolato quanto.
- Un seed per configurazione (due per la migliore): differenze sotto ~0,1–0,3% sono rumore.

## Appendice — Riprodurre

Dalla root della repo, venv attivo. Training set: lo stesso misto GRF + blocchi delle parti 1–2.

```
# solutore e test set (valida contro Ansys, poi genera i .npz)
python new_experiment/fd_solver.py --validate
python new_experiment/make_k_testsets.py
# modello generale (run migliore)
python new_experiment/heat_surface4.py --k_mode full --kink_centers 0 --epochs 60000 \
  --decay_steps 1200 --kbranch_hidden 128 --tag _60k
# fine-tune a k fisso dal modello generale
python new_experiment/heat_surface4.py --k_mode fixed --kink_centers 0 --epochs 10000
--decay_steps 600 \
  --lr 3e-4 --init_from results/results_ktrain/kfull_60k/model.eqx \
  --test data/ktest_ansys11.npz,data/ktest_ansys14.npz --tag _ft
```

File nuovi: new\_experiment/models\_k.py, new\_experiment/heat\_surface4.py, new\_experiment/fd\_solver.py, new\_experiment/make\_k\_testsets.py. Risultati e pesi in results/results\_ktrain/. Nota: il run del report è kfull\_60k\_d1200 (dizionario di kink attivo ma ininfluente); il comando sopra riproduce la variante consigliata, senza dizionario.